Dataset Analysis and Visualization Using Big Data Program

Dataset Link: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

Student Name:

Student ID:

Abstract

Customer churn prediction has emerged as an important analytical problem in service-based industries as a need to retain customers in order to increase long-term profitability. This paper deploys the PySpark envelope of big data analytics to analyze the Telco Customer Churn dataset received on Kaggle, which contains 7,043 customer records and 21 customer attributes, such as demographic, service utilization, service contract, and billing attributes. The missing values were filled in through systematic preprocessing, the categorical fields were coded, and numerical features were scaled using Spark-based transformations to facilitate scalability. Engineered feature vectors were used to train logistic regression and random forest models in order to check the effectiveness in churn prediction. The Random Forest model provided an accuracy of 73.62 and an AUC of 0.8223, which is better than the logistic regression. Furthermore, Tableau dashboards were created to gain a visual insight into churn patterns in relation to gender, dependents, type of contract, type of internet service, and behavior of payments. Both machine learning and visual analytics provide insights that indicate the key drivers of churn include month-to-month contracts, fiber optic internet, increased monthly rates, and reduced term length. The paper is rounded off by giving recommendations on improving retention by optimizing strategies.

Table of Contents

Abstract 2

1. Introduction 4

2. Related Work / Background 5

   2.1 Customer Churn Prediction Models 5

   2.2 Big Data and PySpark in Predictive Analytics 6

   2.3 Data Visualisation in Customer Analytics 6

3. Dataset Description and Data Processing 9

   3.1 Dataset Overview 9

   3.2 Data Cleaning 9

   3.3 Feature Engineering 10

   3.4 Label Preparation and Splitting 11

4. Methodology 12

   4.1 PySpark MLlib Framework 12

   4.2 Machine Learning Pipeline 12

   4.3 Logistic Regression Model 13

   4.4 Random Forest Classifier 14

   4.5 Evaluation Metrics 14

5. Experimental Setup 15

6. Results of Models 15

7. Tableau Visualisations 16

8. Result Discussion 20

   8.1 Behavioural Insights from Predictive Models 20

   8.2 Interpretation of Tableau Dashboards 20

   8.3 Business Implications of Findings 21

9. Conclusion and Future Work 21

9.1 Conclusion 21

9.2 Future Work 22

10. Social Impact 22

Reference 24

List of Figures

Figure 1 Spark setup.....5

Figure 2Customer Churn Prediction Models 6

Figure 3 EDA.....8

Figure 4Dataset Overview 9

Figure 5Data Cleaning 10

Figure 6 Feature engineering 11

Figure 7Label Preparation 11

Figure 8PySparkMLlib Framework 12

Figure 9Machine Learning Pipeline 13

Figure 10Logistic Regression Confusion matrix 13

Figure 11Random Forest Classifier 14

Figure 12Evaluation Metrics 15

Figure 13Churn Distribution Across Gender 16

Figure 14Churn Distribution Across Dependents 17

Figure 15Churn Distribution Across Contract Type 17

Figure 16Churn Distribution Across Internet Services 18

Figure 17Actual Churn Reason Distribution 19

Figure 18Impact of Tenure on Churn 19

Figure 19Impact of Average Monthly Charges on Churn 20

1. Introduction

The issue of customer churn poses a great threat to industries that are based on subscription, especially telecommunication, where competition is quite high and customer acquisition is very expensive. Churn prediction will enable organizations to incorporate specific retention strategies, minimize loss of revenue, and enhance customer satisfaction in the long run. As massive customer data is increased, big data analytics have become a fundamental texture of processing and analysis of large volumes of data effectively, e.g., PySpark. This study explores the dynamics of customer churning by using the Telco Customer Churn dataset whereby PySpark-based machine learning and Tableau visual analytics are combined. The dataset comprises behavioral, demographic, and contractual attributes that determine the churn propensity. This paper seeks to develop an end-to-end analytical workflow, where the starting point will be preprocessing of data by predictive modelling and visual interpolation. Two machine learning algorithms are adopted, namely, logistic regression and random forest, to evaluate the level of predictive performance. It also discusses how the trends of visual images enrich the customer patterns as well as provide a full picture of the causes of customer churn.

```
Installing OpenJDK (if not already installed)...
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list entry mis
Selecting previously unselected package openjdk-11-jre-headless:amd64.
(Reading database ... 121713 files and directories currently installed.)
Preparing to unpack .../openjdk-11-jre-headless_11.0.29+7-1ubuntu1~22.04_amd64.deb ...
Unpacking openjdk-11-jre-headless:amd64 (11.0.29+7-1ubuntu1~22.04) ...
Setting up openjdk-11-jre-headless:amd64 (11.0.29+7-1ubuntu1~22.04) ...
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jjs to provide /usr/bin/jjs (jjs) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/rmid to provide /usr/bin/rmid (rmid) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/pack200 to provide /usr/bin/pack200 (pack200) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/unpack200 to provide /usr/bin/unpack200 (unpack200) in auto mode
JAVA_HOME set to: /usr/lib/jvm/java-11-openjdk-amd64

Installing pyspark==3.5.1 ... (this may take ~30-60s)
java -version:
 ['openjdk version "17.0.16" 2025-07-15', 'OpenJDK Runtime Environment (build 17.0.16+8-Ubuntu-0ubuntu122.04.1)']

=== Environment & Spark summary ===
pyspark (python pkg) version: 3.5.1
spark.version (SparkSession): 3.5.1
Hadoop version (via Spark JVM): 3.3.4
Spark session created successfully.
```

Figure 1 Spark setup

2. Related Work / Background

2.1 Customer Churn Prediction Models

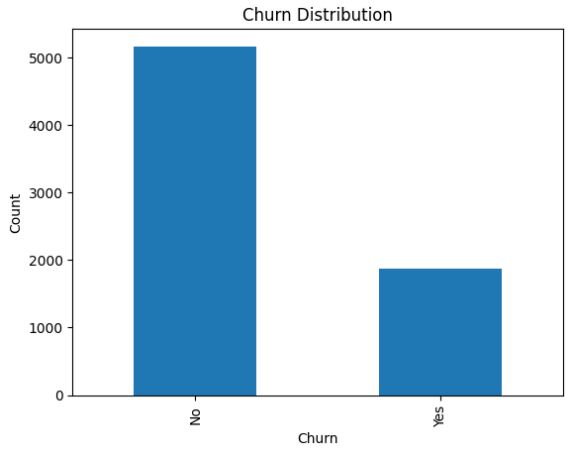
[redacted]

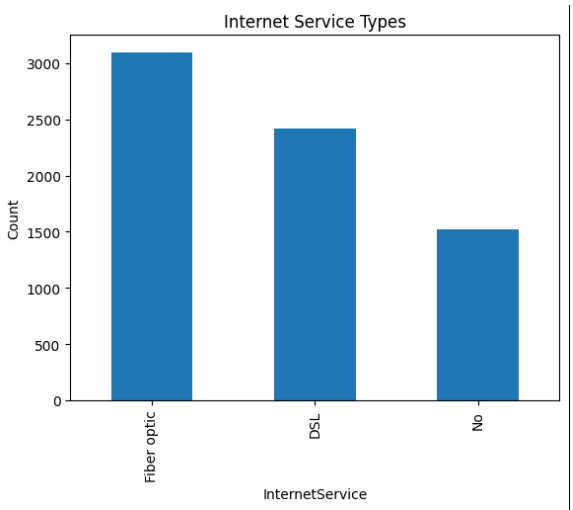
```python
# Kaggle creds (no upload)
import os, json, stat

os.environ['KAGGLE_CONFIG_DIR'] = '/root/.kaggle'
os.makedirs('/root/.kaggle', exist_ok=True)

kaggle_token = {
  "username": "your_username",
  "key": "your_api_key"
}

with open('/root/.kaggle/kaggle.json', 'w') as f:
    json.dump(kaggle_token, f)

os.chmod('/root/.kaggle/kaggle.json', stat.S_IRUSR | stat.S_IWUSR)

print("Kaggle credentials saved to /root/.kaggle/kaggle.json")
```

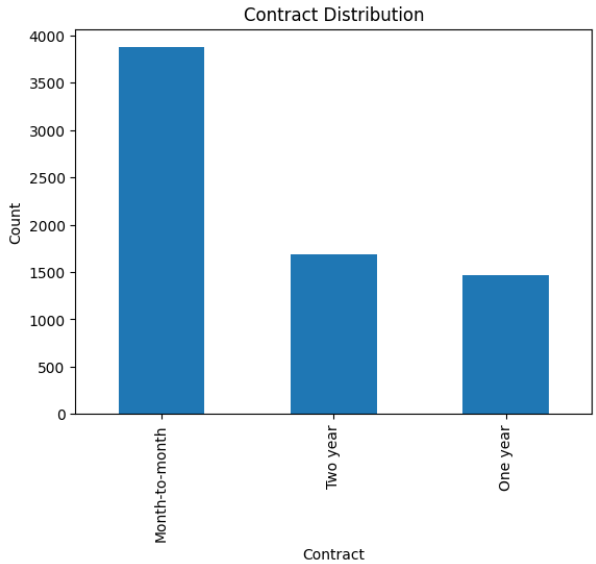
```
Kaggle credentials saved to /root/.kaggle/kaggle.json
```

[redacted]

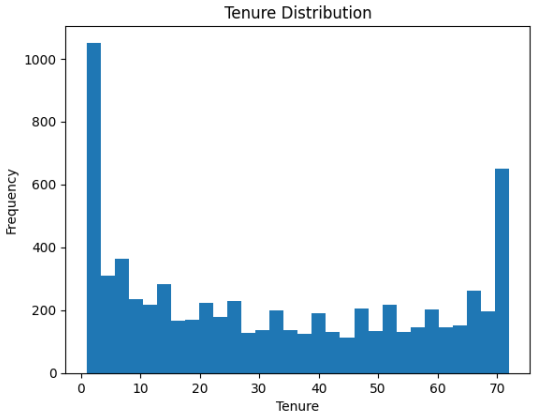
[redacted]

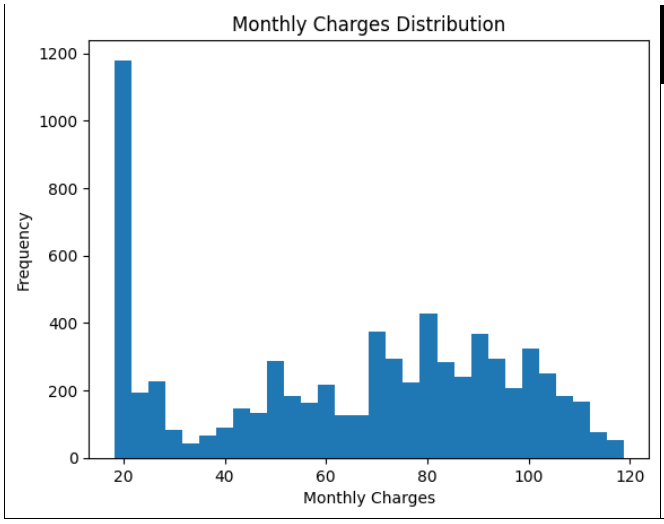




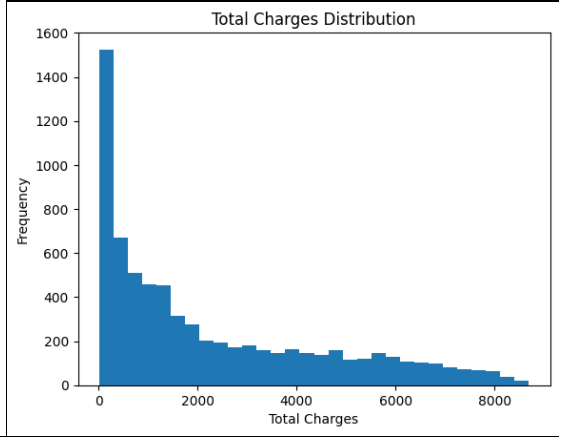


[redacted]





[redacted]



[redacted]

```python
print("Initial row count:", df.count())
print("Columns:", df.columns)

# 1) Trim spaces
df = df.withColumn("TotalCharges", F.trim(F.col("TotalCharges")))

# 2) Convert TotalCharges "" → null then cast
df = df.withColumn(
        "TotalCharges",
        F.when(F.col("TotalCharges") == "", None).otherwise(F.col("TotalCharges"
    ).withColumn("TotalCharges", F.col("TotalCharges").cast("double"))

# 3) Drop rows where TotalCharges is null
df = df.dropna(subset=["TotalCharges"])

df.printSchema()
df.show(5)
```

```
Initial row count: 7043
Columns: ['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure'
root
 |-- customerID: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- SeniorCitizen: integer (nullable = true)
 |-- Partner: string (nullable = true)
 |-- Dependents: string (nullable = true)
 |-- tenure: integer (nullable = true)
 |-- PhoneService: string (nullable = true)
 |-- MultipleLines: string (nullable = true)
 |-- InternetService: string (nullable = true)
 |-- OnlineSecurity: string (nullable = true)
 |-- OnlineBackup: string (nullable = true)
 |-- DeviceProtection: string (nullable = true)
```

```python
# 5 — CLEANING (FIXED)
from pyspark.sql import functions as F

print("Initial row count:", df.count())
print("Columns:", df.columns)

# 1) Trim spaces
df = df.withColumn("TotalCharges", F.trim(F.col("TotalCharges")))

# 2) Convert TotalCharges "" → null then cast
df = df.withColumn(
        "TotalCharges",
        F.when(F.col("TotalCharges") == "", None).otherwise(F.col("TotalCharges"))
    ).withColumn("TotalCharges", F.col("TotalCharges").cast("double"))

# 3) Drop rows where TotalCharges is null
df = df.dropna(subset=["TotalCharges"])

df.printSchema()
df.show(5)
```

```
Initial row count: 7043
Columns: ['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneSe
root
 |-- customerID: string (nullable = true)
 |-- gender: string (nullable = true)
```

```
categorical_cols = [
    "gender","Partner","Dependents","PhoneService","MultipleLines",
    "InternetService","OnlineSecurity","OnlineBackup","DeviceProtection",
    "TechSupport","StreamingTV","StreamingMovies","Contract",
    "PaperlessBilling","PaymentMethod"
]

numeric_cols = ["SeniorCitizen","tenure","MonthlyCharges","TotalCharges"]

# Label indexer
label_indexer = StringIndexer(inputCol=label_col, outputCol="label", handleInvalid="keep")

# Categorical indexers
indexers = [StringIndexer(inputCol=c, outputCol=f"{c}_idx", handleInvalid="keep") for c in categorical_cols]

print("Label + indexers defined.")
```

```
abel + indexers defined.
```

```
# 3: OneHotEncoders
encoders = [OneHotEncoder(inputCols=[f"{c}_idx"], outputCols=[f"{c}_vec"]) for c in categorical_cols]
print("Encoders defined for:", [f"{c}_vec" for c in categorical_cols])
```

```
ncoders defined for: ['gender_vec', 'Partner_vec', 'Dependents_vec', 'PhoneService_vec', 'MultipleLines_vec', 'InternetServ
```

```
# 4: Vector Assembler
feature_cols = [f"{c}_vec" for c in categorical_cols] + numeric_cols

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features", handleInvalid="keep")
print("Assembler will use", len(feature_cols), "inputs.")
```

```
Assembler will use 19 inputs.
```

```
# 5: Build & fit pipeline, transform df -> final_df
pipeline = Pipeline(stages=[label_indexer] + indexers + encoders + [assembler])
pipeline_model = pipeline.fit(df)
final_df = pipeline_model.transform(df)

# quick checks
print("final_df columns snapshot:")
print(final_df.columns[-10:])   # show last columns to confirm
print("Rows:", final_df.count())
final_df.select("label").distinct().show()
```

```
final_df columns snapshot:
['OnlineSecurity_vec', 'OnlineBackup_vec', 'DeviceProtection_vec', 'TechSupport_vec', 'StreamingTV_vec', 'Str
Rows: 7032
+-----+
|label|
+-----+
|  0.0|
|  1.0|
```

[redacted]

[redacted]

[redacted]

```python
# 1. Import required libraries

from pyspark.ml.classification import LogisticRegression
from pyspark.sql.functions import col
from pyspark.mllib.evaluation import BinaryClassificationMetrics

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns


# 2. Fit Logistic Regression model

lr = LogisticRegression(featuresCol="features", labelCol="label", maxIter=20)
lr_model = lr.fit(train_df)


# 3. Predict on test data

pred_lr = lr_model.transform(test_df)


# 4. Calculate AUC

score_and_labels_lr = pred_lr.select("probability", "label") \
    .rdd.map(lambda row: (float(row.probability[1]), float(row.label)))
```

[redacted]

[redacted]

[redacted]

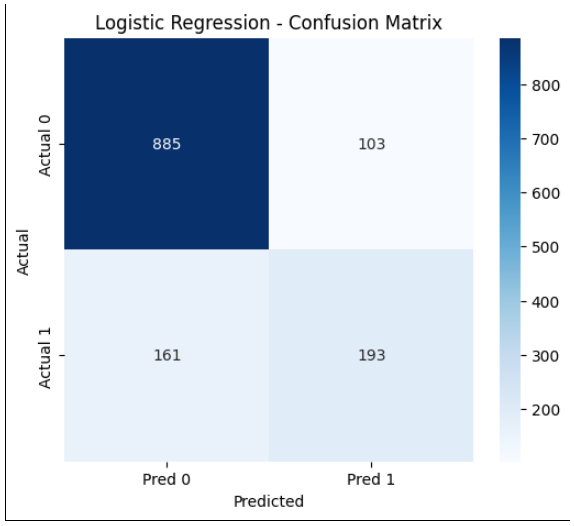
```python
# 5: Build & fit pipeline, transform df -> final_df
pipeline = Pipeline(stages=[label_indexer] + indexers + encoders + [assembler])
pipeline_model = pipeline.fit(df)
final_df = pipeline_model.transform(df)

# quick checks
print("final_df columns snapshot:")
print(final_df.columns[-10:])   # show last columns to confirm
print("Rows:", final_df.count())
final_df.select("label").distinct().show()
```

```
final_df columns snapshot:
['OnlineSecurity_vec', 'OnlineBackup_vec', 'DeviceProtection_vec', 'TechSupport_vec', 'StreamingT
Rows: 7032
+-----+
|label|
+-----+
|  0.0|
|  1.0|
+-----+
```

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

```
LR Confusion Matrix:
+-----+----------+-----+
|label|prediction|count|
+-----+----------+-----+
|  0.0|       0.0|  885|
|  0.0|       1.0|  103|
|  1.0|       0.0|  161|
|  1.0|       1.0|  193|
+-----+----------+-----+

RF Confusion Matrix:
+-----+----------+-----+
|label|prediction|count|
+-----+----------+-----+
|  0.0|       0.0|  988|
|  1.0|       0.0|  354|
+-----+----------+-----+
```

[redacted]

[redacted]

[redacted]

```
/usr/local/lib/python3.12/dist-packages/pyspark/s
  warnings.warn(
Logistic Regression AUC: 0.8556405681740243

Confusion Matrix:
[[885 103]
 [161 193]]
```

```
/usr/local/lib/python3.12/dist-packages/pyspark/sql/c
  warnings.warn(
Random Forest AUC: 0.8223426885335896
Random Forest Accuracy: 0.736214605067064
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

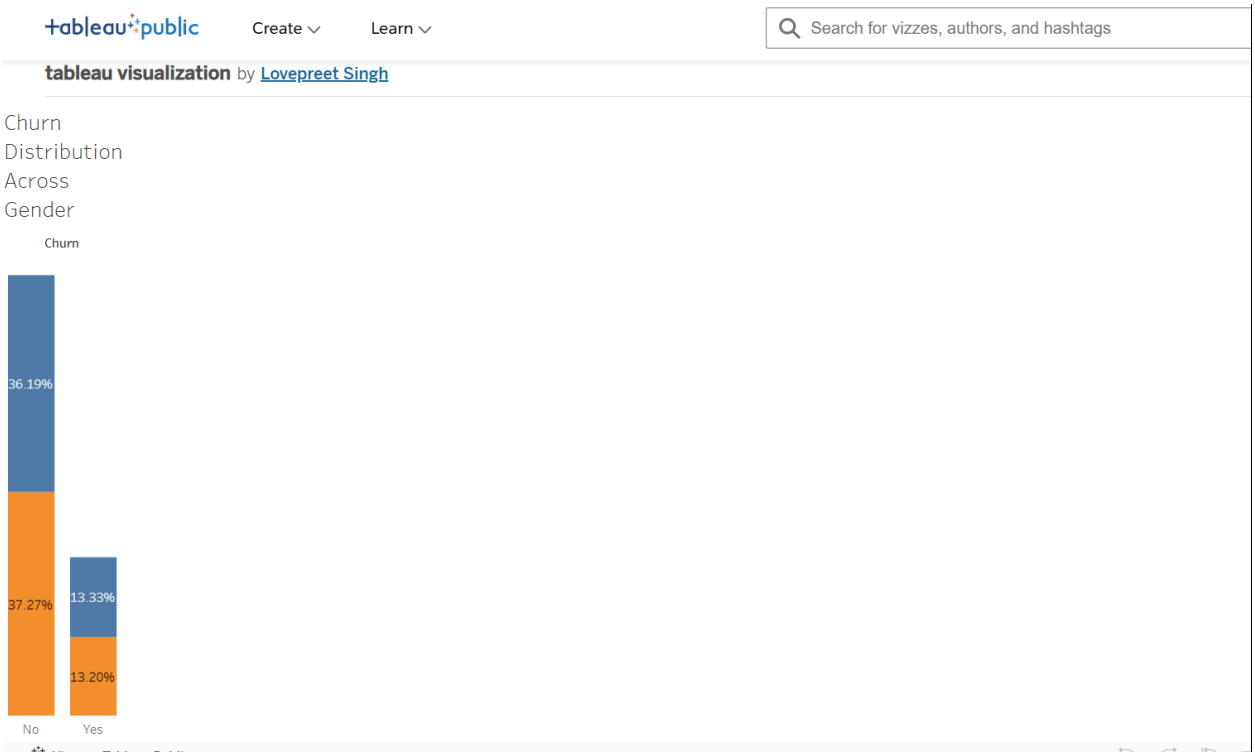
[redacted]

[redacted]

[redacted]

[redacted]

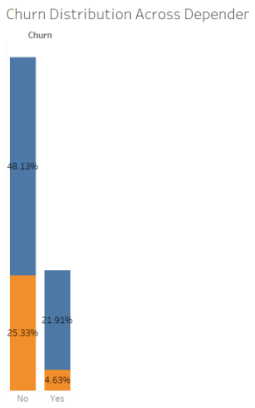
[redacted]



[redacted]

[redacted]

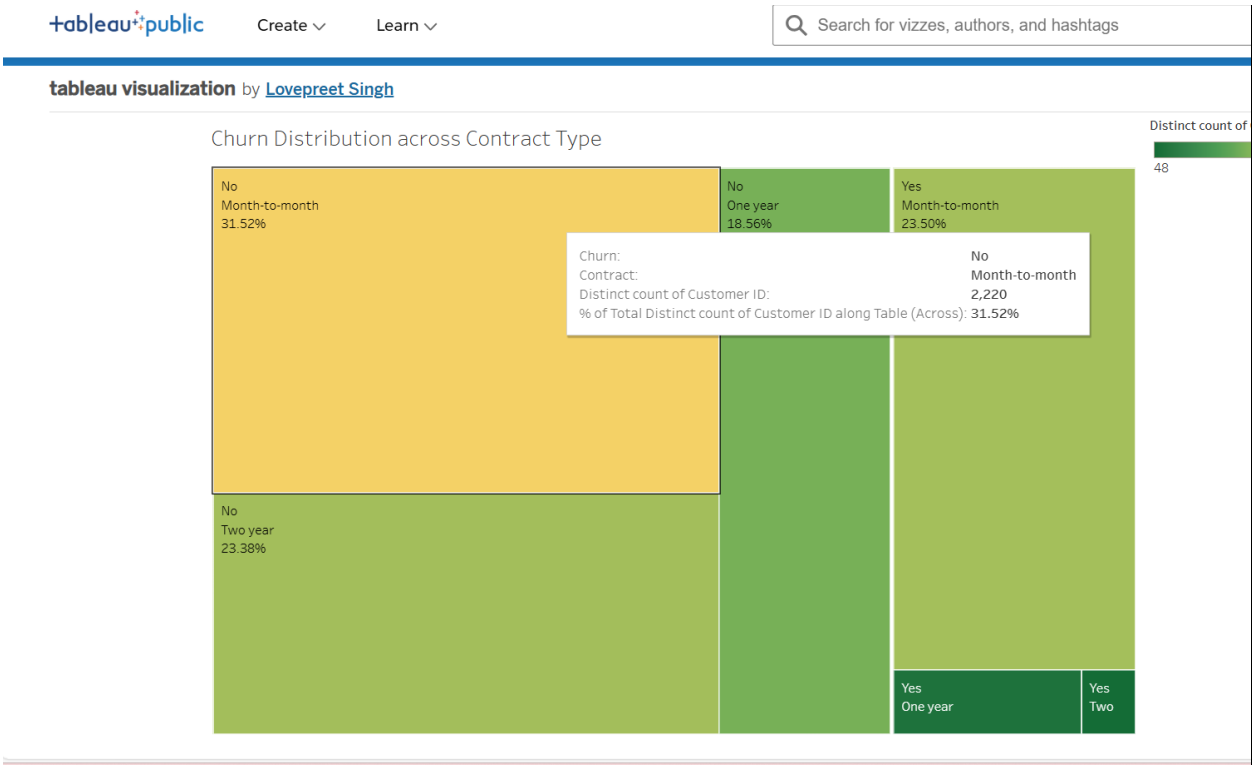
[redacted]



[redacted]

[redacted]

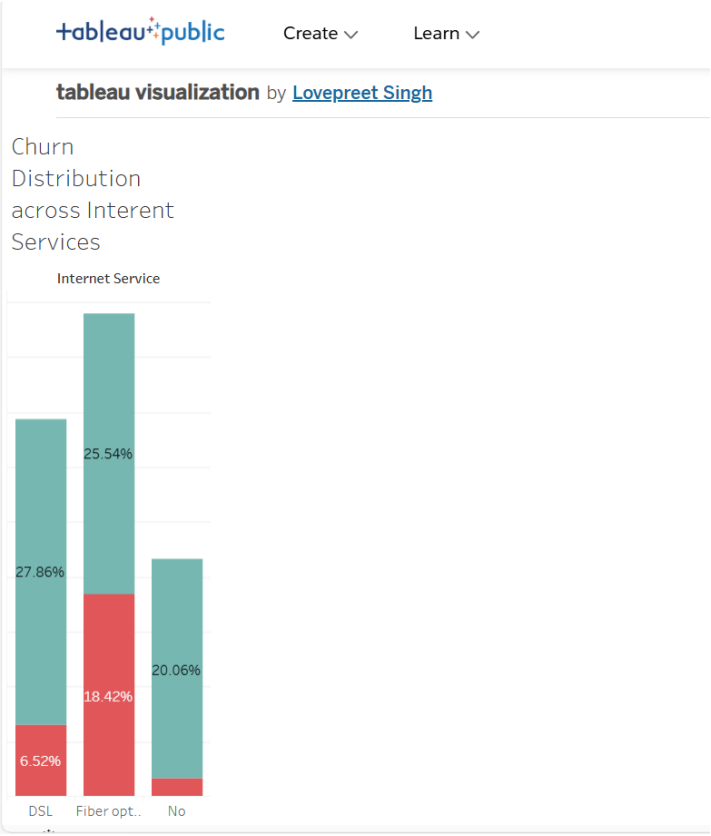
[redacted]



[redacted]

[redacted]

[redacted]



[redacted]

tableau visualization by [Lovepreet Singh]

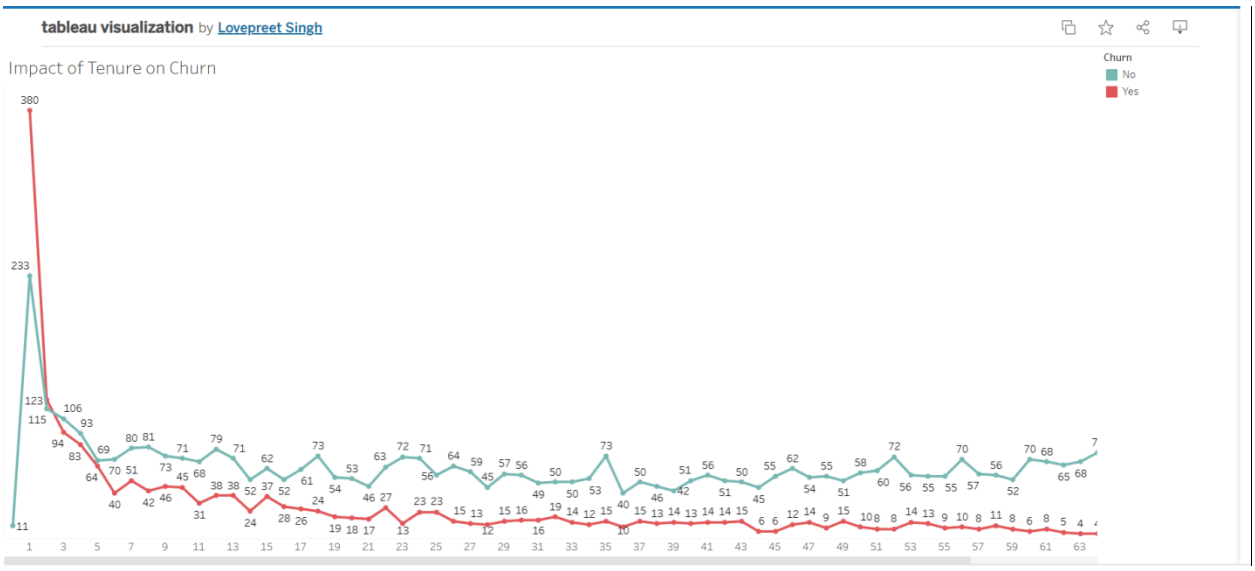
Actual Churn Reason Distribution

			Online Security / Streaming Movies				
			No		No internet ..	Yes	
Partner	Streaming TV	Tech Support	No	Yes	No internet ..	No	Yes
No	No	No	20.76%	6.47%		2.84%	0.64%
		Yes	2.30%	0.75%		0.80%	0.37%
	No internet service	No internet service			4.49%		
	Yes	No	6.31%	11.24%		0.70%	1.50%
		Yes	0.91%	2.62%		0.27%	1.23%
Yes	No	No	7.28%	3.64%		1.66%	0.64%
		Yes	1.02%	0.48%		0.54%	0.21%
	No internet service	No internet service			1.55%		
	Yes	No	3.05%	8.13%		0.64%	1.87%
		Yes	0.64%	2.57%		0.48%	1.39%

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

impact of average monthly charges on Churn

Multiple Lines

Churn	No	No ph..	Yes
No	49.9	43.5	79.2
Yes	67.2	37.5	89.1

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
- [redacted]
  - [redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]